Project Scope — Multi-Task ADMET (v0.1)

1) Purpose

Build a lean, defensible **multi-task ADMET predictor** that shows rigor (fair splits, calibration, ablations) and plugs directly into your 30-day pocket-edit proof as a **developability filter**. Target completion: **3–4 days**.

2) Problem statement

Given a molecule (SMILES), predict a small panel of ADMET endpoints using a **shared molecular encoder** with **task-specific heads**, training with **label masking** to exploit partially labeled data.

Target tasks

- **Classification (4):** BBBP, hERG, CYP3A4 (inhibitory), ClinTox *(fallback: Tox21 if ClinTox slows you down)*
- **Regression (1):** ESOL (logS) *(optional +1: FreeSolv)*

3) Success criteria (Definition of Done)

- **Data & splits**
  - Deterministic **Murcko scaffold** split persisted for each task (train/val/test).
  - Leakage check passes (no shared scaffolds across splits).
- **Model quality**
  - **Macro-AUC (test)** across classification tasks **≥ single-task macro-AUC + 0.01**.
  - At least **one tox task ECE ≤ 0.10** after temperature scaling.
  - **ESOL RMSE ≤ 0.8–1.0** on test.
- **Scientific reporting**
  - **One table**: per-task AUROC/AUPRC/RMSE (val & test) + macro averages.
  - **Two plots**: (i) per-task metrics bar chart, (ii) reliability diagram for the noisiest tox task.
  - **README ≤ 300 words** with 2 crisp insights and exact split protocol.
- **Integration hook**
  - CLI to score a CSV of SMILES and emit **calibrated task probabilities** + PASS/FAIL flags for thresholds (val-tuned). This will be used by the pocket-edit project.

4) Deliverables

- `admet-multitask/` repo with:
  - Code: `feats/rdkit.py`, `models/mt_mlp.py`, `loaders/tdc.py`, `train.py`, `eval.py`
  - Data artifacts: cached features, **split indices**
  - Reports: `results_val_test.csv`, figures, `README.md`

- Repro: `config.yaml`, `requirements.txt`, seed fixed
- CLI: `python eval.py --smiles_file candidates.csv --tasks BBBP,HERG,... --calibrated`

5) Datasets & features

- **Datasets:** BBBP, hERG, CYP3A4, ClinTox (or Tox21), ESOL (all standard, small-to-medium).
- **Primary features:** RDKit **ECFP4 (1024 bits)**, radius 2.
- **Optional descriptors (Day 3 if time allows):** TPSA, LogP, MW, HBD, HBA, rotatable bonds (20–30 scalar features concatenated).

6) Model & training

- **Encoder (shared trunk):**
  `1024 → Dense(512) → ReLU → Dropout(0.3) → Dense(256) → ReLU → Dropout(0.2)`
- **Heads:**
  - Classification: `Linear(256→1)` → logits; loss: **BCEWithLogits** (+ `pos_weight` per task)
  - Regression: `Linear(256→1)`; loss: **MSE**
- **Optimizer: AdamW** (`lr=3e-4`, `wd=1e-4`), batch **256**, epochs **≤50**, **early stopping** on **macro-AUC (val)** (patience 5).
- **Label masking:** build dense `targets[task]` and `masks[task]`; compute each task loss only where `mask==1`. Total loss = mean over available task losses per batch.
- **Class imbalance:** compute `pos_weight` from **train** split; if a task is still unstable, switch that single task to **focal loss** (γ=1.5) and document.

7) Splits & evaluation protocol

- **Split strategy:** For each dataset **before any featurization**, compute **Murcko scaffold**; create **75/15/10** train/val/test split. Persist indices to `data/splits/{task}_{seed}.json`.
- **Multi-task alignment:** Build a union index over SMILES; tasks may be missing labels → use masks.
- **Metrics**
  - **Classification:** AUROC, **AUPRC** (emphasize this), F1@val-tuned threshold (report on test).
  - **Regression:** RMSE, MAE; optional **Concordance Index**.
  - **Calibration: Temperature scaling** on **val logits**; report **ECE (15 bins)** + reliability diagram.

8) Experiments / ablations (must-have, minimal)

1. **Single-task vs Multi-task (ECFP-MLP)** — same trunk width; compare per-task metrics (test).
2. **Random vs Scaffold split** *(on one classification task)* — show the expected performance drop to argue realism.

3. **(Optional)** Uncertainty-weighted multi-task (homoscedastic): learn σ^2 per task; objective `L/σ² + log σ`.

Stop rule: Do **not** expand tasks beyond the listed set; do **not** hyper-sweep—change one variable per ablation.

9) Timeline (3–4 days)

- **Day 1 — Data, features, splits, baselines**
  - Implement loaders; compute ECFP; cache features (`.npz` keyed by canonical SMILES).
  - Create/persist scaffold splits; leakage check (no shared scaffolds).
  - Baselines: Logistic Regression (cls) & Ridge (reg); save `reports/baselines.csv`.
- **Day 2 — Multi-task model + masking + first results**
  - Implement trunk + heads + masked loss.
  - Train with early stopping; save logits/probs for calibration.
  - Export `results_val_test.csv` (per-task metrics).
- **Day 3 — Calibration + ablations + plots + README**
  - Fit temperature scaling per cls task; recompute test metrics & **ECE**.
  - Run 2 ablations (single vs multi; random vs scaffold on one task).
  - Generate metrics bar chart + one reliability diagram.
  - Write **README (≤300 words)** with exact protocol & 2 insights.
- **Day 4 (buffer) — Small polish**
  - Optional uncertainty weighting **or** add small descriptor block to features.
  - Implement **CLI** scorer for integration; tag **v0.1**.

10) Risks & mitigations

- **Data leakage via splits:** enforce Murcko before split; unit test: verify no scaffold overlap.
- **Imbalance hurting AUPRC:** use `pos_weight` from **train only**; optionally focal on that single task.
- **Masking bugs:** unit test masked loss with synthetic inputs; assert `mask.sum()>0` per task/batch.
- **Over-tuning / scope creep:** freeze config on Day 2; prioritize reporting & calibration over model tweaks.

11) Repo structure

```
lua
CopyEdit
admet-multitask/
  data/
    raw/ processed/ splits/
  feats/
    rdkit.py
  loaders/
    tdc.py
  models/
```

```
    mt_mlp.py
  common/
    metrics.py   # AUROC, AUPRC, ECE, RMSE, CI
    splits.py    # Murcko, random
    utils.py
  train.py
  eval.py
  config.yaml
  reports/
    baselines.csv
    results_val_test.csv
    metrics_bar.png
    reliability_HERG.png
  README.md
  requirements.txt
```

12) Configuration (authoritative knobs)

```
yaml
CopyEdit
seed: 42
features: {type: ecfp4, nbits: 1024, add_descriptors: false}
tasks:
  classification: [BBBP, HERG, CYP3A4, ClinTox]
  regression: [ESOL]
splits: {type: murcko, ratios: [0.75, 0.15, 0.10]}
train:
  batch_size: 256
  epochs: 50
  lr: 3e-4
  weight_decay: 1e-4
  dropout: [0.3, 0.2]
loss:
  cls: bce_logits
  pos_weight: auto
  focal_overrides: []
early_stopping: {metric: macro_auc_cls, patience: 5}
calibration: {temperature_scaling: true, bins: 15}
```

13) Reporting checklist (before tagging v0.1)

- `baselines.csv` with single-task results
- `results_val_test.csv` with multi-task results (val & test)
- `metrics_bar.png` (AUROC/AUPRC and RMSE)
- `reliability_<task>.png` with ECE number in caption
- README with: tasks, features, split protocol, metrics table, 2 insights, and **how to reproduce**
- CLI scoring example & thresholds (from val)

14) Out of scope (v0.1)

- GNN encoders (GIN/MPNN), data augmentation, transfer learning, ensembles, uncertainty via MC-dropout, per-task hyper-sweeps, and external ADMET datasets. *(These are candidates for v0.2 if time remains.)*

15) Future extensions (post-v0.1, optional)

- Swap trunk to **GIN** with simple atom/bond features; compare vs ECFP-MLP.
- **GradNorm** or dynamic loss weighting; analyze task interference (gradient cosine).
- **MC-dropout** or ensembles; threshold by uncertainty for safer screening.
- **Calibration per chemical region** (scaffold or logP bins) to show reliability stratification.